

# Unique Binary Search Trees

LeetCode **96. Unique Binary Search Trees** is one of the most important Dynamic Programming problems because it introduces the **Catalan Number** concept.

There are **4 approaches** to solve it:

1. Brute Force Recursion
2. Recursion + Memoization (Top-Down DP)
3. Bottom-Up DP (Tabulation)
4. Catalan Number Formula (Mathematical)

I'll explain them in the order you should learn them.

## Problem Statement

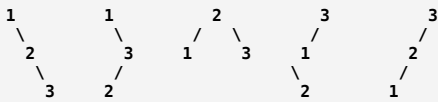
Given  $n$  distinct numbers  $1 \dots n$ , count how many **different BSTs** can be formed.

Example

**Input:**  $n = 3$

**Output:** 5

The BSTs are



Answer = 5

## Step 1 : Understand the BST Property

Suppose

1 2 3 4 5

If you choose

3

as root,

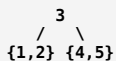
everything smaller goes left

1 2

everything larger goes right

4 5

So



Now,

the left subtree itself is another BST problem.

The right subtree is also another BST problem.

This is the key observation.

## Step 2 : Think about every possible root

For

1 2 3

Possible roots are

```
1
2
3
```

We try every root.

## Root = 1

```
1
 \
 2 3
```

Left subtree

```
empty
```

Right subtree

```
2 3
```

Ways

```
Left = 1
Right = ?
```

## Root = 2

```
  2
 / \
1   3
```

Left

```
1
```

Right

```
3
```

## Root = 3

```
  3
 /
1 2
```

Immediately we notice

The answer is

```
Left Ways × Right Ways
```

because

every left BST can pair with every right BST.

This multiplication is the whole DP.

## Why Multiply?

Suppose

Left has

```
2 BSTs
```

Right has

```
3 BSTs
```

Then

```
Left1 Right1
Left1 Right2
Left1 Right3

Left2 Right1
Left2 Right2
Left2 Right3
```

Total

```
2 × 3 = 6
```

## General Formula

Suppose

```
root = i
```

Then

Left side contains

```
i-1 nodes
```

Right side contains

```
n-i nodes
```

If

```
leftWays = f(i-1)
rightWays = f(n-i)
```

then

```
ways = leftWays × rightWays
```

Now sum over every root

```
f(n) = Σ f(i-1) × f(n-i)
```

## APPROACH 1 : Pure Recursion

Define

```
solve(n)
returns number of BSTs using n nodes.
```

## Base Case

No node

```
Empty Tree
```

There is exactly

```
1
```

empty BST.

Similarly

One node

```
1 BST
```

So

```
n <= 1
return 1
```

## Recursive Formula

Loop

```
root = 1 to n
```

Compute

```
left = solve(root-1)
right = solve(n-root)
answer += left * right
```

## Code

```
C++
class Solution {
public:
    int solve(int n) {
        // Empty tree or single node tree
        if (n <= 1)
            return 1;

        int ans = 0;

        // Try every node as root
        for (int root = 1; root <= n; root++) {
            int left = solve(root - 1);
            int right = solve(n - root);
            ans += left * right;
        }

        return ans;
    }

    int numTrees(int n) {
        return solve(n);
    }
};
```

## Complexity

Time

Exponential

Many repeated calls.

## Why recursion is slow?

Look at

```
solve(5)
calls
solve(4)
solve(3)
solve(2)
```

Later

```
solve(4)
again calls
solve(3)
solve(2)
```

Again repeated.

## APPROACH 2 : Memoization

Store already computed answers.

Create

```
dp[n]
```

meaning

```
Number of BSTs with n nodes.
```

Before solving

```
solve(n)
```

check

```
if(dp[n] != -1)
return dp[n];
```

## Code

```
C++
class Solution {
public:
    vector<int> dp;
    int solve(int n) {
        if (n <= 1)
            return 1;
        if (dp[n] != -1)
            return dp[n];
        int ans = 0;
        for (int root = 1; root <= n; root++) {
            int left = solve(root - 1);
            int right = solve(n - root);
            ans += left * right;
        }
        return dp[n] = ans;
    }
    int numTrees(int n) {
        dp.resize(n + 1, -1);
        return solve(n);
    }
};
```

## Complexity

States

```
n
```

Each state loops

```
n
```

Overall

```
O(n2)
```

Space

```
O(n)
```

## APPROACH 3 : Bottom-Up DP (Best Interview Approach)

Instead of recursion,

build answers from smaller values.

## Meaning

```
dp[i]
=
number of BSTs having i nodes
```

Initialize

```
dp[0]=1
dp[1]=1
```

Now compute

```
dp[2]
dp[3]
...
dp[n]
```

## Dry Run

Suppose

```
n=4
```

Initially

```
dp
0 1 2 3 4
1 1 0 0 0
```

## Compute dp[2]

Possible roots

```
1
2
```

Root 1

```
left=0
right=1
1x1=1
```

Root2

```
left=1
right=0
1x1=1
```

Total

```
2
```

Now

```
dp
1 1 2 0 0
```

## Compute dp[3]

Root1

```
left=0
right=2
```

```
1x2=2
```

Root2

```
left=1
right=1
1x1=1
```

Root3

```
left=2
right=0
2x1=2
```

Total

```
5
```

Now

```
1 1 2 5 0
```

## Compute dp[4]

Root1

```
1x5=5
```

Root2

```
1x2=2
```

Root3

```
2x1=2
```

Root4

```
5x1=5
```

Total

```
14
```

## Code

```
C++
class Solution {
public:
    int numTrees(int n) {
        // dp[i] = Number of unique BSTs using i nodes
        vector<int> dp(n + 1, 0);

        // Base cases:
        // Empty tree has one valid BST.
        dp[0] = 1;

        // Single node has one BST.
        dp[1] = 1;

        // Compute answers from 2 nodes up to n nodes
        for (int nodes = 2; nodes <= n; nodes++) {
            // Try every node as the root
            for (int root = 1; root <= nodes; root++) {
                // Nodes in left subtree
                int leftNodes = root - 1;

                // Nodes in right subtree
                int rightNodes = nodes - root;

                // Combine every left BST with every right BST
                dp[nodes] += dp[leftNodes] * dp[rightNodes];
            }
        }

        return dp[n];
    }
};
```

```
};
}
```

## Complexity

Time

$O(n^2)$

Space

$O(n)$

## APPROACH 4 : Catalan Number

The recurrence

$f(n) = \sum_{i=0}^{n-1} f(i) \times f(n-i-1)$

is exactly the recurrence of the **Catalan Numbers**.

The closed-form formula is:

$$C_n = \frac{(2n)!}{(n+1)! \cdot n!}$$

This can also be written as:

$$C_n = \frac{\binom{2n}{n}}{n+1}$$

For example:

| n | Answer |
|---|--------|
| 0 | 1      |
| 1 | 1      |
| 2 | 2      |
| 3 | 5      |
| 4 | 14     |
| 5 | 42     |
| 6 | 132    |

Because factorials grow very quickly, a multiplicative formula is preferred.

```
C++
class Solution {
public:
    int numTrees(int n) {
        long long catalan = 1;

        for (int i = 0; i < n; i++) {
            catalan = catalan * (2LL * n - i) / (i + 1);
        }

        return catalan / (n + 1);
    }
};
```

**Time:**  $O(n)$

**Space:**  $O(1)$

## Why does this DP work? (Intuition)

Suppose you have  $n$  nodes.

1. Pick one node as the root.
2. The left subtree must use all smaller values.
3. The right subtree must use all larger values.
4. The **actual values don't matter**—only how many nodes go left and right. A BST built from  $\{1, 2\}$  has the same number of shapes as one built from  $\{4, 5\}$ .
5. Left and right subtree choices are independent, so multiply their counts.



6. Sum over every possible root.

This gives:

$$dp[n] = \sum_{root=1}^n dp[root-1] \times dp[n-root]$$

## Summary

| Approach        | Idea                              | Time               | Space          |
|-----------------|-----------------------------------|--------------------|----------------|
| Recursion       | Try every root recursively        | Exponential        | O(n) recursion |
| Memoization     | Cache repeated subproblems        | O(n <sup>2</sup> ) | O(n)           |
| Bottom-Up DP    | Build answers from 0 to n         | O(n <sup>2</sup> ) | O(n)           |
| Catalan Formula | Closed-form mathematical solution | O(n)               | O(1)           |

## Interview recommendation

In an interview, present the approaches in this order:

1. Explain the BST decomposition and derive the recurrence.
2. Write the recursive solution.
3. Optimize it with memoization.
4. Convert it to bottom-up DP (this is the expected coding solution).
5. Mention that the recurrence is the Catalan recurrence, and if asked, discuss the mathematical formula as an optimization.

ChatGPT can make mistakes. Check important info.